

Milestone 3

API Sprint 1 Consolidated Report

Project: Community Services Platform

System: Smart Library Management System

Team: MAY2026-Team-021

Submission date: 2 August 2026

Team Members

1. Shubham Nagar (21f3000093)
2. Gaurav Ramesh Ghodge (22f2000467)
3. Piyush Jha (21f1002373)
4. Mandeep (23f2001951)
5. Hitarth Mehta (23f1001948)

1. Sprint Summary

Sprint 1 replaced the frontend-only mock foundation with working FastAPI, SQLAlchemy, JWT, tenant isolation, and PostgreSQL-compatible domain APIs. The result is an integrated owner workflow for registering a library, managing students and physical resources, allocating seats, recording fees, publishing announcements, and viewing operational reports.

74

documented API operations

133 passed

backend pytest cases in the final run

87

backend application Python files

26

backend test modules

Sprint goals

1. Establish a secure authentication, role, tenant, error, and API-contract foundation.
2. Implement owner-facing core library, seat-allocation, payment, and announcement workflows.
3. Integrate completed frontend services with real APIs without changing the page/store/service architecture.
4. Protect historical records, prevent cross-library access, and test positive and negative paths.
5. Generate a Swagger-compatible YAML contract with operation descriptions, errors, and user-story mapping.

Final technical baseline

- FastAPI routes use Pydantic schemas, service rules, tenant-scoped repositories, and SQLAlchemy models.
- Passwords use salted **scrypt** hashes. Access tokens are JWTs and refresh sessions rotate and are revocable.
- Every expected API error uses a structured code/message/details envelope and includes an `X-Request-ID`.
- Browser authentication uses an HttpOnly refresh cookie; API clients may use explicit refresh tokens.
- Frontend integrations retain the Vue page to Pinia store to Axios service flow.

2. Architecture and Functional Delivery

```
Vue page/component
-> Pinia store
-> Axios service
-> FastAPI route + authentication/tenant dependency
-> Pydantic request/response schema
-> application service + transaction boundary
-> tenant-scoped SQLAlchemy repository
-> database models
```

Functional area	Delivered behavior
Authentication	Library registration, login, access/refresh tokens, refresh rotation, browser sessions, logout, current user, profile, password change, and invitation activation.
Core library data	Student CRUD/status/invitations, floor CRUD, physical seat CRUD and bulk status, shift CRUD and overlap validation, and library settings.
Seat allocation	Date- and shift-aware availability, overnight overlap detection, atomic multi-shift creation, conflict prevention, lifecycle closure, snapshots, and history.
Fees and payments	Idempotent monthly generation, full/partial transactions, calculated balances, receipts, downloads, and persisted reminder-link generation.
Announcements	Owner list/create/update/publish/archive/delete lifecycle and student feed/read state.
Seat requests	Student submit/cancel workflow and tenant-scoped owner review.
Dashboard and reports	Database-backed student, seat, occupancy, revenue, pending-fee, ageing, activity, and attention metrics.
Student self-service	Own profile, seat/allocation history, fee summary, receipts, announcements, and seat-change requests.

Important business rules implemented

- The active membership determines `libraryId` ; tenant-owned APIs never trust a client-supplied library identifier.
- Maintenance, blocked, inactive, or deleted seats and inactive/deleted shifts cannot receive new allocations.
- Allocation date ranges are inclusive; consecutive booking starts on the day after the earlier allocation ends.
- Shift comparison handles midnight crossings such as 22:00-02:00 versus 01:00-06:00.
- Monthly fee records are idempotent and payment balances are calculated from completed append-only transactions.
- Soft deletion and lifecycle statuses retain student, seat, shift, allocation, payment, receipt, and audit history.

3. API Inventory and Swagger Evidence

The submitted `openapi.yaml` is generated from FastAPI and validated against OpenAPI 3.1. It contains stable operation IDs, descriptions, request/response schemas, authentication requirements, structured error responses, request-ID headers, examples, and `x-user-stories` mappings.

Route group	Operations	Representative user stories
Authentication	13	Register library, login, rotate session, logout, profile, password, student activation
Students	9	Owner CRUD and invitation; student profile view/update
Floors and seats	12	Physical layout CRUD, status and bulk actions
Shifts	6	Shift administration and overlapping-selection validation
Seat allocations	5	Availability, create, list/history, close, student seat view
Seat requests	5	Student submit/cancel; owner list/review
Payments and receipts	9	Monthly generation, transactions, receipts, reminder links, student fee view
Announcements	8	Owner lifecycle; student audience feed and read state
Reports	3	Filter options, owner reports, operational dashboard
Settings and system	4	Library settings, API discovery, health
Total	74	All operations mapped in the YAML

Final contract audit: 74 operations; 0 missing descriptions; 0 missing user-story mappings; 0 missing documented 4xx/5xx responses. The semantic drift test passed against the checked-in YAML.

Shared error contract

```
{
  "error": {
    "code": "SEAT_ALLOCATION_CONFLICT",
    "message": "Seat A-01 is occupied during the requested period.",
    "details": {}
  },
  "requestId": "frontend-request-123"
}
```

4. API Test Cases - Part A

Environment: Python 3.12, FastAPI in-process ASGI client, and an isolated test database.

API-001 - Register library with initial physical resources

API	POST /api/v1/auth/register-library
Input	Valid library/owner details with <code>seatCount: 25</code> .
Expected	201; admin user, library ID, access/refresh tokens; one settings row, Floor 1, three default shifts, and 25 seats.
Actual	201; all response fields and database bootstrap counts matched.
Result	SUCCESS
Pytest	<code>test_registration_bootstraps_library_and_me</code>

Snapshot of pytest function - backend/tests/test_auth_api.py

```
16 def test_registration_bootstraps_library_and_me(api) -> None:
17     client, testing_session = api
18
19     response = register(client, seatCount=25)
20
21     assert response.status_code == 201
22     body = response.json()
23     assert body["user"]["role"] == "admin"
24     assert body["user"]["libraryId"]
25     assert body["accessToken"]
26     assert body["refreshToken"]
27
28     me = client.get(
29         "/api/v1/auth/me",
30         headers=bearer(body["accessToken"]),
31     )
32     assert me.status_code == 200
33     assert me.json()["email"] == "owner@example.com"
34
35     with testing_session() as db:
36         assert db.scalar(select(func.count()).select_from(LibrarySettings)) == 1
37         assert db.scalar(select(func.count()).select_from(Floor)) == 1
38         assert db.scalar(select(func.count()).select_from(Shift)) == 3
39         assert db.scalar(select(func.count()).select_from(Seat)) == 25
```

API-002 - Reject invalid registration seat count

API	POST /api/v1/auth/register-library
Input	Valid registration fields with <code>seatCount: 0</code> .
Expected	422 with error code <code>VALIDATION_ERROR</code> , field details, and matching request ID.
Actual	422; message was "The request contains invalid data." and request ID matched the response header.
Result	SUCCESS
Pytest	<code>test_validation_errors_use_the_shared_error_contract</code>

Snapshot of pytest function - backend/tests/test_api_foundation.py

```
66 def test_validation_errors_use_the_shared_error_contract(api) -> None:
67     client, _testing_session = api
68
69     response = register(client, seatCount=0)
70
71     assert response.status_code == 422
72     body = response.json()
73     assert body["error"]["code"] == "VALIDATION_ERROR"
74     assert body["error"]["message"] == "The request contains invalid data."
75     assert body["error"]["details"]["fields"]
76     assert body["requestId"] == response.headers["x-request-id"]
```

API-003 - Refresh-token rotation and reuse prevention

API	POST /api/v1/auth/refresh
Input	Use a valid refresh token, then submit the original token again after rotation.
Expected	First request 200 with new tokens; reused old refresh token 401; old access session revoked.
Actual	200 for rotation; 401 for token reuse and old access-token use.
Result	SUCCESS
Pytest	test_refresh_rotation_revokes_old_session_and_logout_revokes_new_one

Snapshot of pytest function - backend/tests/test_auth_api.py

```
55 def test_refresh_rotation_revokes_old_session_and_logout_revokes_new_one(api) -> None:
56     client, _testing_session = api
57     registered = register(client).json()
58
59     refreshed = client.post(
60         "/api/v1/auth/refresh",
61         json={"refreshToken": registered["refreshToken"]},
62     )
63     assert refreshed.status_code == 200
64     rotated = refreshed.json()
65     assert rotated["refreshToken"] != registered["refreshToken"]
66
67     reused = client.post(
68         "/api/v1/auth/refresh",
69         json={"refreshToken": registered["refreshToken"]},
70     )
71     assert reused.status_code == 401
72
73     old_access = client.get(
74         "/api/v1/auth/me",
75         headers=bearer(registered["accessToken"]),
76     )
77     assert old_access.status_code == 401
```

API-004 - Student invitation activation and one-time use

APIs	POST /students, GET /auth/invitations/validate, POST /auth/invitations/accept
Input	Create student with invitation, validate token, set password, then reuse the same token.
Expected	Student 201; validation 200; activation 200; repeated activation rejected with 422.
Actual	All statuses matched; activated credentials logged in as the Student role.
Result	SUCCESS
Pytest	test_student_crud_invitation_and_password_setup

Snapshot of pytest function - backend/tests/test_core_library_api.py

```
24 def test_student_crud_invitation_and_password_setup(api, owner_headers) -> None:
25     client, _ = api
26     created = client.post(
27         "/api/v1/students",
28         json=student_payload(),
29         headers=owner_headers,
30     )
31     assert created.status_code == 201
32     student = created.json()["data"]
33     assert student["enrollmentNumber"].startswith("STU-")
34     assert student["portalAccessStatus"] == "not_activated"
35     setup_url = student["invitationSetupUrl"]
36     token = parse_qs(urlsplit(setup_url).query)["token"][0]
37
38     validation = client.get(
39         "/api/v1/auth/invitations/validate",
40         params={"token": token},
41     )
42     assert validation.status_code == 200
43     assert validation.json()["email"] == "aarav@example.com"
44
45     accepted = client.post(
46         "/api/v1/auth/invitations/accept",
47         json={"token": token, "password": "StudentPass123"},
48     )
49     assert accepted.status_code == 200
50     assert client.post(
51         "/api/v1/auth/invitations/accept",
52         json={"token": token, "password": "StudentPass123"},
53     ).status_code == 422
54
55     login = client.post(
56         "/api/v1/auth/session/login",
57         json={
58             "email": "aarav@example.com",
59             "password": "StudentPass123",
60         },
61     )
62     assert login.status_code == 200
63     assert login.json()["user"]["role"] == "student"
```

5. API Test Cases - Part B

API-005 - Detect overlapping overnight shifts

API	POST /api/v1/shifts/validate-selection
Input	Overnight 22:00-02:00 and Early 01:00-06:00 shift IDs.
Expected	200 response with <code>data.valid: false</code> because the shifts overlap from 01:00 to 02:00.
Actual	200 with invalid selection and overlap conflict details.
Result	SUCCESS
Pytest	<code>test_floor_seat_shift_and_settings_workflows</code>

Snapshot of relevant pytest function lines - backend/tests/test_core_library_api.py

```
171     overnight = client.post(
172         "/api/v1/shifts",
173         json={
174             "name": "Overnight",
175             "startTime": "22:00",
176             "endTime": "02:00",
177         },
178         headers=owner_headers,
179     )
180     early = client.post(
181         "/api/v1/shifts",
182         json={
183             "name": "Early",
184             "startTime": "01:00",
185             "endTime": "06:00",
186         },
187         headers=owner_headers,
188     )
189     assert overnight.status_code == 201
190     assert overnight.json()["data"]["crossesMidnight"] is True
191     overlap = client.post(
192         "/api/v1/shifts/validate-selection",
193         json={
194             "shiftIds": [
195                 overnight.json()["data"]["id"],
196                 early.json()["data"]["id"],
197             ]
198         },
199         headers=owner_headers,
200     )
201     assert overlap.status_code == 200
202     assert overlap.json()["data"]["valid"] is False
```

API-006 - Reject overlapping seat allocation

API	POST /api/v1/seat-allocations
Input	A second student requests the same seat and Morning shift during an overlapping date range.
Expected	409 with SEAT_ALLOCATION_CONFLICT ; no partial allocation or success audit record.
Actual	409 with the expected code; database and audit counts remained unchanged.
Result	SUCCESS
Pytest	test_conflicts_atomicity_status_precedence_and_consecutive_dates

Snapshot of relevant pytest function lines - backend/tests/test_seat_allocation_api.py

```
176 def test_conflicts_atomicity_status_precedence_and_consecutive_dates(  
177     api,  
178     owner_headers,  
179 ) -> None:  
180     client, testing_session = api  
181     seats, shifts = _resources(client, owner_headers)  
182     morning = next(shift for shift in shifts if shift["name"] == "Morning")  
183     afternoon = next(shift for shift in shifts if shift["name"] == "Afternoon")  
184     first_student = _create_student(client, owner_headers)  
185     second_student = _create_student(  
186         client,  
187         owner_headers,  
188         firstName="Ananya",  
189         lastName="Das",  
190         email="ananya@example.com",  
191         phone="9876543213",  
192     )  
193     first_end = date.today() + timedelta(days=10)  
194  
195     first = client.post(  
196         "/api/v1/seat-allocations",  
197         json=_allocation_payload(  
198             first_student["id"],  
199             seats[0]["id"],  
200             [morning["id"]],  
201             endDate=first_end.isoformat(),  
202         ),  
203         headers=owner_headers,  
204     )  
205     assert first.status_code == 201  
206  
207     seat_conflict = client.post(  
208         "/api/v1/seat-allocations",  
209         json=_allocation_payload(  
210             second_student["id"],  
211             seats[0]["id"],  
212             [morning["id"]],  
213         ),  
214         headers=owner_headers,  
215     )  
216     assert seat_conflict.status_code == 409  
217     assert seat_conflict.json()["error"]["code"] == "SEAT_ALLOCATION_CONFLICT"
```

API-007 - Record partial and final payment

API	POST /api/v1/payments/{feeRecordId}/transactions
Input	Fee total 1200.00; transactions of 400.25 and 799.75.
Expected	First response partially paid with 799.75 balance; final response paid with 0.00 balance and two completed transactions.
Actual	Calculated totals, lifecycle status, transaction count, receipt count, and audit count matched.
Result	SUCCESS
Pytest	test_partial_and_full_payments_recalculate_server_totals

Snapshot of pytest function - partial payment assertions

```
129 def test_partial_and_full_payments_recalculate_server_totals(  
130     api,  
131     owner_headers,  
132 ) -> None:  
133     client, testing_session = api  
134     _create_student(client, owner_headers, feeAmount=1200)  
135     fee = _generate(client, owner_headers).json()["data"]  
136         "createdPayments"  
137     ][0]  
138  
139     partial = client.post(  
140         f"/api/v1/payments/{fee['id']}/transactions",  
141         json={  
142             "amount": "400.25",  
143             "method": "upi",  
144             "referenceNumber": "UPI-PARTIAL-001",  
145             "notes": "First instalment",  
146         },  
147         headers={**owner_headers, "X-Request-ID": "payment-partial-test"},  
148     )  
149     assert partial.status_code == 201  
150     assert partial.headers["x-request-id"] == "payment-partial-test"  
151     payment = partial.json()["data"]["payment"]  
152     assert payment["paidAmount"] == "400.25"  
153     assert payment["balanceAmount"] == "799.75"  
154     assert payment["status"] == "partially_paid"
```

Snapshot of pytest function - final payment assertions

```
172     completed = client.post(  
173         f"/api/v1/payments/{fee['id']}/transactions",  
174         json={  
175             "amount": "799.75",  
176             "method": "cash",  
177             "referenceNumber": "CASH-FINAL-001",  
178         },  
179         headers=owner_headers,  
180     )  
181     assert completed.status_code == 201  
182     paid = completed.json()["data"]["payment"]  
183     assert paid["paidAmount"] == "1200.00"  
184     assert paid["balanceAmount"] == "0.00"  
185     assert paid["status"] == "paid"  
186     assert len(paid["transactions"]) == 2
```

API-008 - Hide another library's student

API	GET /api/v1/students/{studentId}
Input	Owner B requests a Student ID belonging to Library A.
Expected	404, without exposing the student name or Library A.
Actual	404; cross-library student record was hidden.
Result	SUCCESS
Pytest	test_student_records_are_isolated_by_library

Snapshot of pytest function - backend/tests/test_core_library_api.py

```
102 def test_student_records_are_isolated_by_library(api, owner_headers) -> None:
103     client, _ = api
104     created = client.post(
105         "/api/v1/students",
106         json=student_payload(sendInvitation=False),
107         headers=owner_headers,
108     )
109     student_id = created.json()["data"]["id"]
110
111     second_registration = register(
112         client,
113         libraryName="Second Library",
114         email="second.owner@example.com",
115         phone="9876543299",
116     )
117     second_headers = bearer(second_registration.json()["accessToken"])
118     hidden = client.get(
119         f"/api/v1/students/{student_id}",
120         headers=second_headers,
121     )
122     assert hidden.status_code == 404
```

6. Required Expected-vs-Actual Mismatch

API-009 - Expired database session initially accepted an access JWT

API	GET /api/v1/auth/me
Input	A cryptographically valid, unexpired access JWT whose backing <code>user_sessions.expires_at</code> value was changed to a past time.
Expected	401 Unauthorized because the server-side session is expired.
Initial actual	200 OK . The signed JWT was accepted even though its backing database session had expired.
Initial result	FAIL
Root cause	The authentication dependency validated JWT claims but did not enforce the backing session's expiry/revocation state for every protected request.
Fix	Added active-session validation to protected requests and strengthened refresh/logout revocation. Implemented in commit 3dd6ad9 .
Retest actual	401 Unauthorized.
Retest result	SUCCESS
Pytest	<code>test_expired_backing_session_is_rejected</code>

Snapshot of the regression test that initially failed and now passes - `backend/tests/test_auth_api.py`

```
98 def test_expired_backing_session_is_rejected(api) -> None:
99     client, testing_session = api
100     registered = register(client).json()
101     claims = decode_access_token(registered["accessToken"])
102     assert claims is not None
103
104     with testing_session() as db:
105         session = db.get(UserSession, uuid.UUID(claims["sid"]))
106         assert session is not None
107         session.expires_at = datetime.now(timezone.utc) - timedelta(minutes=1)
108         db.commit()
109
110     response = client.get(
111         "/api/v1/auth/me",
112         headers=bearer(registered["accessToken"]),
113     )
114     assert response.status_code == 401
```

This mismatch demonstrates the value of API testing: signature validation alone looked correct during normal login testing, but a server-side session expiry scenario revealed that revoked or expired sessions could remain usable. The regression test now prevents recurrence.

Other defects found and resolved during Sprint 1

Defect	Resolution	Evidence
Concurrent refresh or payment/allocation requests could race.	Added deterministic row locking, atomic transactions, conflict handling, and rollback tests.	Concurrency-focused API tests and service-level transaction tests.
Registration did not initially bootstrap all requested library resources.	Registration now creates settings, Floor 1, default shifts, requested seats, membership, owner, and session atomically.	<code>test_registration_bootstraps_library_and_me</code>
Payment list parameters temporarily drifted from the frontend/API contract.	Restored the complete parameter set and regenerated the checked-in OpenAPI file.	Commit <code>838a0af</code> and semantic drift test.
Overnight shifts could be mishandled by simple linear time comparison.	Split crossing-midnight intervals and reused one authoritative overlap algorithm in validation and allocation checks.	Overnight unit and API tests.

7. Test Execution and Quality Evidence

Command/check	Final result
<code>backend/venv/bin/pytest -q</code>	133 backend tests passed
<code>python backend/scripts/export_openapi.py --check</code>	OpenAPI contract valid and current
<code>pytest backend/tests/test_openapi_contract.py</code>	3 passed, including all-operation documentation audit
OpenAPI completeness scan	74 operations; no missing descriptions, stories, or error responses
Frontend service/component tests	Passed during integrated module validation
Frontend lint and production build	Passed during integrated module validation

Test categories represented

- Successful creation, retrieval, update, status, lifecycle, and deletion paths.
- Required-field and business-rule validation.
- Unauthenticated, malformed-token, wrong-role, inactive membership, and suspended-library access.
- Cross-tenant resource hiding and not-found behavior.
- Duplicate records, payment overrun, seat/student conflicts, and transaction rollback.
- Search, filters, pagination, sorting, date overlap, overnight overlap, ageing boundaries, and report reconciliation.
- OpenAPI validity and semantic drift between FastAPI and the submitted YAML.

8. User Feedback and Actions

The integrated owner workflows were demonstrated and reviewed by team members acting in the Library Owner role. Feedback was captured during the Seat Management and Payment Management walkthroughs and then converted into fixes or backend acceptance tests.

Feedback received	Decision	Action taken
An overnight shift (22:00-02:00) and early shift (01:00-06:00) must be shown as overlapping.	Fix immediately	Implemented midnight-aware overlap logic and added frontend and backend regression tests.
Seat availability should be calculated for the selected shift instead of showing every shift inside every seat card.	Accepted	Created a shift-first seat map and one authoritative availability API with blocker details.
Student registration/edit should allow multiple non-overlapping shifts and only show seats available for those shifts.	Accepted	Added multi-shift validation, real availability lookup, and atomic student/allocation changes with rollback.
Action menus should close after an action, outside click, or Escape, and clickable controls should show a pointer cursor.	Fix immediately	Added a reusable dismissible-menu pattern and consistent trigger behavior.
Monthly payments should open on the current month, allow other months, and give owners access to receipts.	Accepted	Connected month-filtered fee APIs, receipt list/detail/download, and payment history.
API failures should explain conflicts clearly instead of exposing database errors.	Accepted	Introduced typed business errors, stable codes, safe messages, field details, and request correlation IDs.

What users considered successful

- Shift-aware seat status made the same physical seat understandable across Morning, Afternoon, Evening, and custom shifts.
- Student registration, seat assignment, and payment workflows remained usable on mobile layouts.
- Clear conflict messages and retained allocation/payment history improved administrative confidence.

9. Plan for Sprint 2

Several Milestone 4 vertical slices were completed early, including student self-service, seat-request review, receipts/reminders, and owner reports. Sprint 2 will focus on the remaining platform scope and release hardening.

Priority	Planned work	Exit evidence
1	Implement Super Admin library, owner, platform analytics, and platform-settings APIs and replace remaining production mocks.	OpenAPI routes, role/tenant tests, frontend integration, reviewed PR.
2	Run concurrency and report-performance suites against a disposable test database.	Passing tests, query plans, timings, and evidence-backed index decision.
3	Expand audit coverage for student, floor, shift, and settings mutations and add an authorized audit-history view if time permits.	Audit assertions and tenant/role tests.
4	Complete scheduled/background behavior for fee generation, reminders, and announcement lifecycle where required by final scope.	Idempotency tests and documented task execution.
5	Conduct end-to-end owner and student UAT, browser/accessibility regression, security configuration review, and deployment rehearsal.	Feedback log, resolved issues, screenshots, test report, deployment checklist.
6	Prepare Milestone 4 submission and final demonstration.	Updated PDF, Swagger YAML, executable code ZIP, test evidence, tagged release.

Known Sprint 2 risks

- A dedicated disposable test database is required for destructive locking and performance evidence.
- Super Admin frontend currently has mock-backed areas and must not silently fall back after API integration.
- Final deployment secrets, secure cookies, CORS origins, and database migrations require environment-specific verification.

10. Submission Contents and Reproduction

A. Milestone 3 PDF Report

This document contains sprint delivery, API test cases, the required failed test, user feedback, fixes, and the next-sprint plan.

B. Swagger YAML

`openapi.yaml` documents all 74 current operations with schemas, descriptions, user stories, errors, security requirements, and request-ID headers.

C. Backend ZIP

The ZIP contains backend application code, migrations, tests, scripts, dependency manifest, example environment, documentation, CI workflow, and a copy of the API contract. It intentionally excludes the local `.env`, virtual environment, caches, bytecode, databases, and secrets.

Local execution

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
pytest
uvicorn app.main:app --reload
```

Swagger verification

```
python scripts/export_openapi.py --check
pytest tests/test_openapi_contract.py
# Swagger UI: http://127.0.0.1:8000/docs
```

Final status: report prepared from repository evidence; Swagger contract current; backend standard test suite passing; executable submission package assembled without secrets.

Evidence references: commits `3dd6ad9`, `ecc3fdb`, `ac4ea6f`, `5202f0d`, `0cec5ba`, `885ceb7`, `4471fbd`, `802c0d6`, `f39a1bb`, `c22a17f`, and `838a0af`.